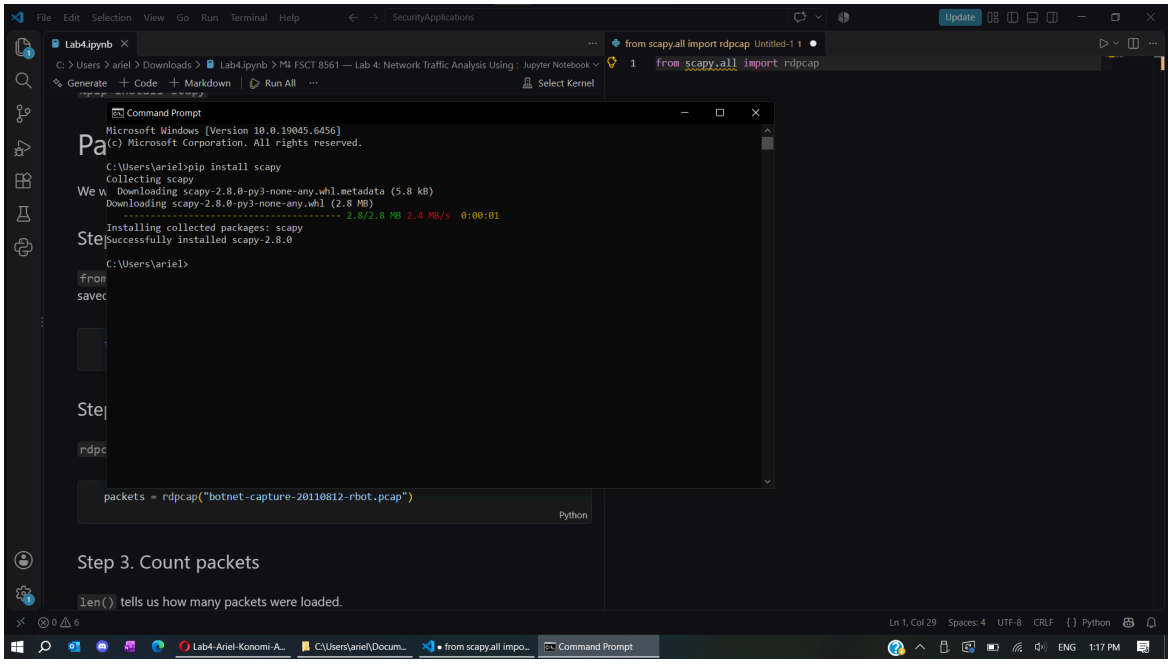
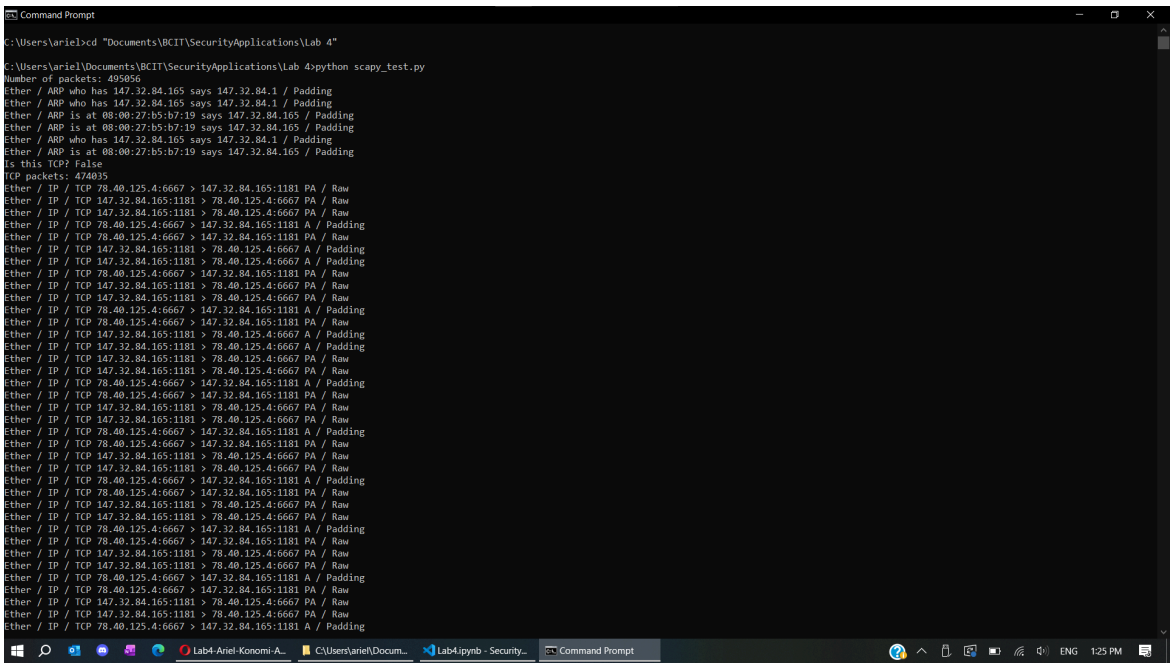
Lab 4: Network Traffic Analysis Using Scapy

Part 1





Step 3

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 4>python scapy_test.py
Number of packets: 495056
```

Step 4

```
Ether / ARP who has 147.32.84.165 says 147.32.84.1 / Padding
```

Step 5

```
Ether / ARP who has 147.32.84.165 says 147.32.84.1 / Padding
Ether / ARP is at 08:00:27:b5:b7:19 says 147.32.84.165 / Padding
Ether / ARP is at 08:00:27:b5:b7:19 says 147.32.84.165 / Padding
Ether / ARP who has 147.32.84.165 says 147.32.84.1 / Padding
Ether / ARP is at 08:00:27:b5:b7:19 says 147.32.84.165 / Padding
```

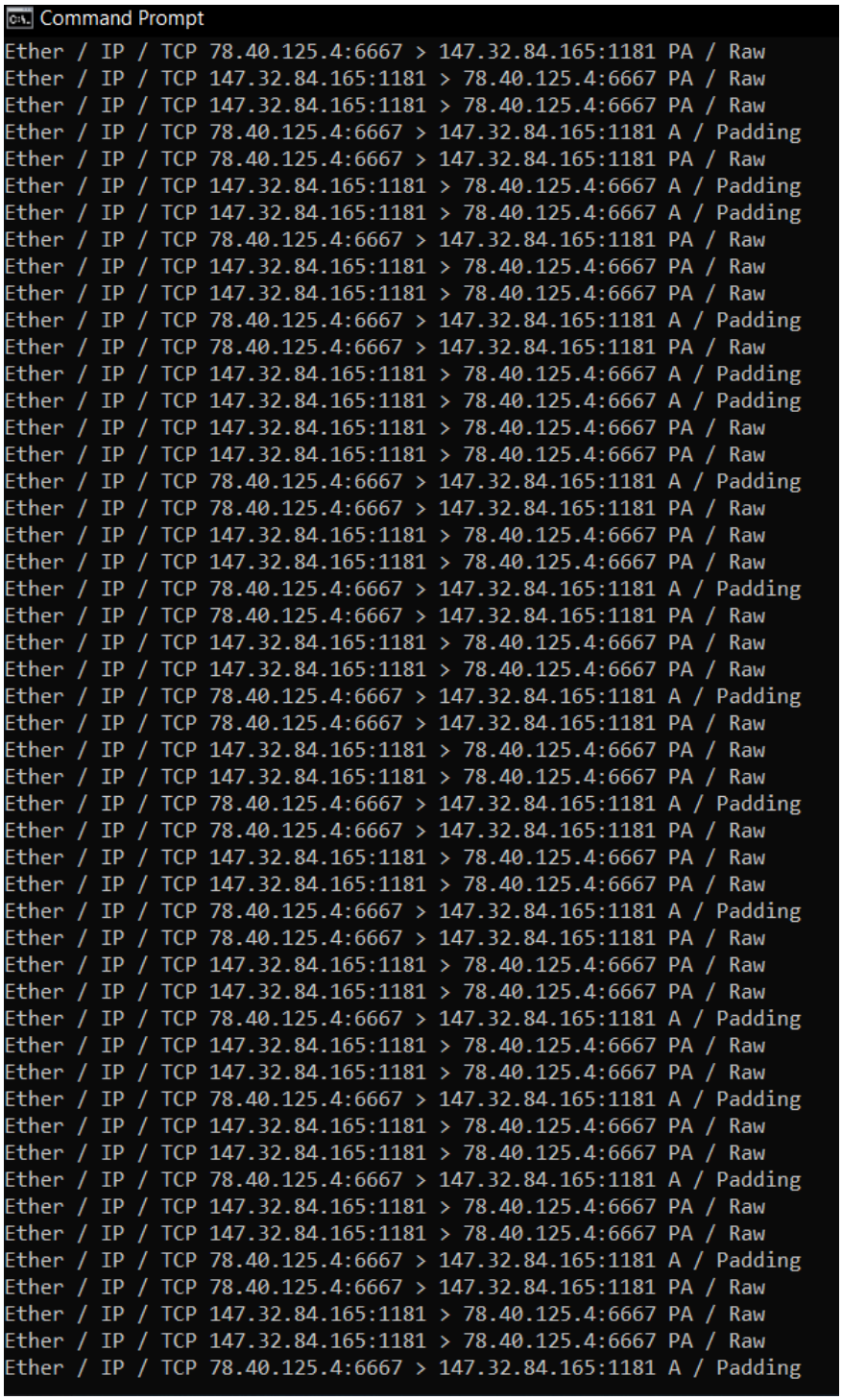
Step 6

```
Is this TCP? False
```

Step 7

```
TCP packets: 474035
```

Step 8


Command Prompt
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 A / Padding
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 A / Padding
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 A / Padding
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 A / Padding
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding

Step 9

```
Source port: 6667
Destination port: 1181
```

```
Source port: 6667
Destination port: 1181
TCP port 80 packets: 22430
Ether / IP / TCP 147.32.84.165:1183 > 111.89.136.28:http S
Ether / IP / TCP 147.32.84.165:1183 > 111.89.136.28:http S
Ether / IP / TCP 147.32.84.165:1184 > 111.89.136.29:http S
Ether / IP / TCP 147.32.84.165:1184 > 111.89.136.29:http S
Ether / IP / TCP 147.32.84.165:1185 > 111.89.136.30:http S
Ether / IP / TCP 147.32.84.165:1185 > 111.89.136.30:http S
Ether / IP / TCP 147.32.84.165:1186 > 111.89.136.31:http S
Ether / IP / TCP 147.32.84.165:1186 > 111.89.136.31:http S
Ether / IP / TCP 147.32.84.165:1187 > 111.89.136.32:http S
Ether / IP / TCP 147.32.84.165:1187 > 111.89.136.32:http S
Ether / IP / TCP 147.32.84.165:1188 > 111.89.136.33:http S
Ether / IP / TCP 147.32.84.165:1188 > 111.89.136.33:http S
Ether / IP / TCP 147.32.84.165:1189 > 111.89.136.34:http S
Ether / IP / TCP 147.32.84.165:1189 > 111.89.136.34:http S
Ether / IP / TCP 147.32.84.165:1190 > 111.89.136.35:http S
Ether / IP / TCP 147.32.84.165:1190 > 111.89.136.35:http S
Ether / IP / TCP 147.32.84.165:1191 > 111.89.136.36:http S
Ether / IP / TCP 147.32.84.165:1191 > 111.89.136.36:http S
Ether / IP / TCP 147.32.84.165:1192 > 111.89.136.37:http S
Ether / IP / TCP 147.32.84.165:1192 > 111.89.136.37:http S
Ether / IP / TCP 111.89.136.28:http > 147.32.84.165:1183 SA
Ether / IP / TCP 147.32.84.165:1193 > 111.89.136.38:http S
Ether / IP / TCP 147.32.84.165:1193 > 111.89.136.38:http S
Ether / IP / TCP 147.32.84.165:1183 > 111.89.136.28:http A / Padding
Ether / IP / TCP 147.32.84.165:1183 > 111.89.136.28:http A / Padding
Ether / IP / TCP 147.32.84.165:1183 > 111.89.136.28:http FA / Padding
Ether / IP / TCP 147.32.84.165:1183 > 111.89.136.28:http FA / Padding
Ether / IP / TCP 111.89.136.29:http > 147.32.84.165:1184 SA
Ether / IP / TCP 147.32.84.165:1194 > 111.89.136.39:http S
Ether / IP / TCP 147.32.84.165:1194 > 111.89.136.39:http S
Ether / IP / TCP 147.32.84.165:1184 > 111.89.136.29:http A / Padding
Ether / IP / TCP 147.32.84.165:1184 > 111.89.136.29:http A / Padding
Ether / IP / TCP 147.32.84.165:1184 > 111.89.136.29:http FA / Padding
Ether / IP / TCP 147.32.84.165:1184 > 111.89.136.29:http FA / Padding
Ether / IP / TCP 111.89.136.31:http > 147.32.84.165:1186 SA
Ether / IP / TCP 147.32.84.165:1195 > 111.89.136.40:http S
Ether / IP / TCP 147.32.84.165:1195 > 111.89.136.40:http S
Ether / IP / TCP 147.32.84.165:1186 > 111.89.136.31:http A / Padding
Ether / IP / TCP 147.32.84.165:1186 > 111.89.136.31:http A / Padding
Ether / IP / TCP 147.32.84.165:1186 > 111.89.136.31:http FA / Padding
Ether / IP / TCP 147.32.84.165:1186 > 111.89.136.31:http FA / Padding
Ether / IP / TCP 111.89.136.30:http > 147.32.84.165:1185 SA
Ether / IP / TCP 147.32.84.165:1196 > 111.89.136.41:http S
Ether / IP / TCP 147.32.84.165:1196 > 111.89.136.41:http S
Ether / IP / TCP 147.32.84.165:1185 > 111.89.136.30:http A / Padding
Ether / IP / TCP 147.32.84.165:1185 > 111.89.136.30:http A / Padding
Ether / IP / TCP 147.32.84.165:1185 > 111.89.136.30:http FA / Padding
Ether / IP / TCP 147.32.84.165:1185 > 111.89.136.30:http FA / Padding
Ether / IP / TCP 111.89.136.33:http > 147.32.84.165:1188 SA
Ether / IP / TCP 111.89.136.32:http > 147.32.84.165:1187 SA
```

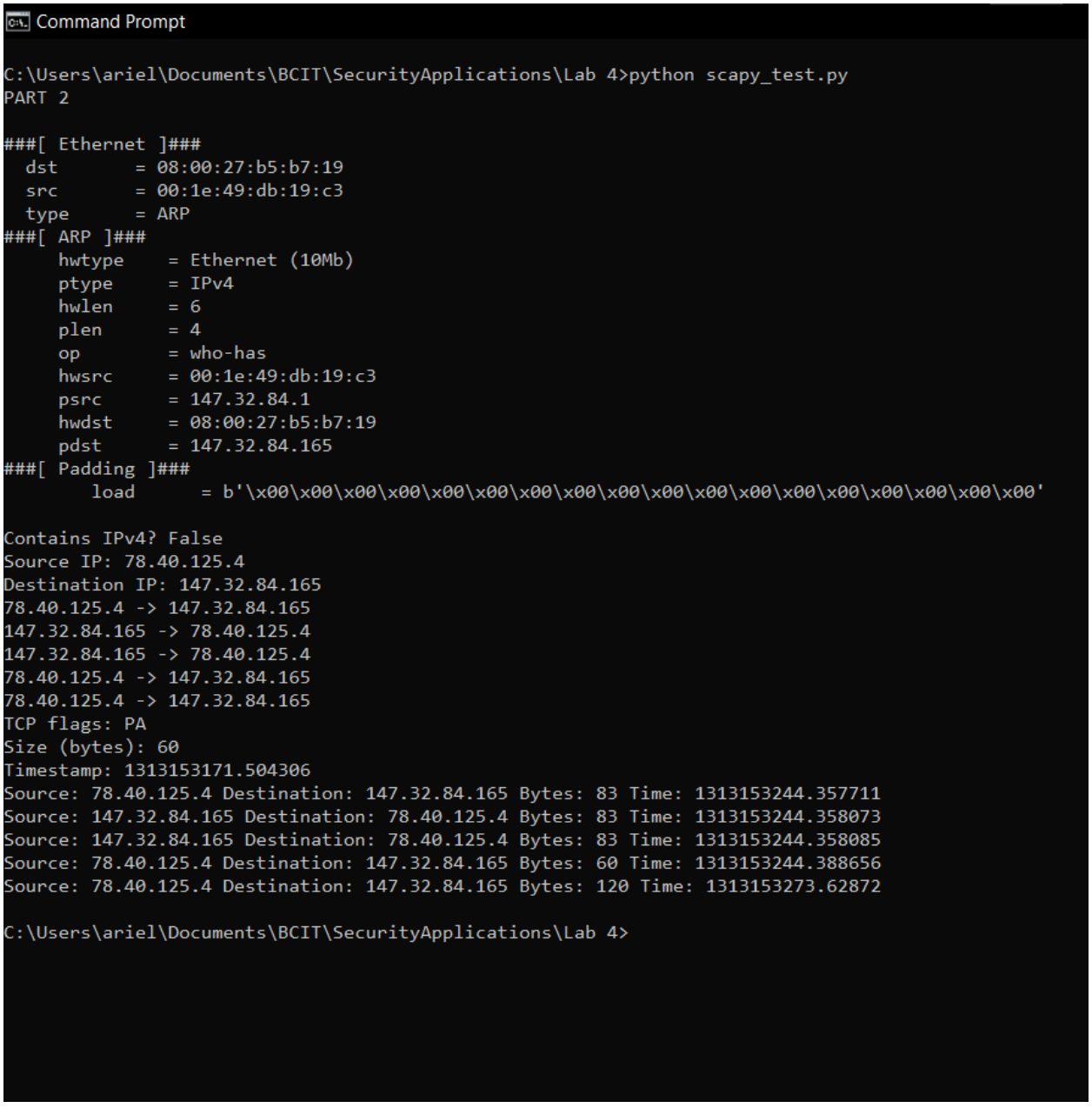
Step 10

```
UDP port 53 packets: 57
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Ans b'chat.freenode.net.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Ans b'chat.freenode.net.'
Ether / IP / UDP / DNS Qry b'wpad.'
Ether / IP / UDP / DNS Qry b'wpad.'
Ether / IP / UDP / DNS Ans name-error
Ether / IP / UDP / DNS Qry b'cr-tools.clients.google.com.'
Ether / IP / UDP / DNS Qry b'cr-tools.clients.google.com.'
Ether / IP / UDP / DNS Ans b'clients.l.google.com.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Ans b'chat.freenode.net.'
Ether / IP / UDP / DNS Qry b'javadl-esd.sun.com.'
Ether / IP / UDP / DNS Qry b'javadl-esd.sun.com.'
Ether / IP / UDP / DNS Ans b'javadl-esd-sino.sun.com.edgesuite.net.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Qry b'wpad.'
Ether / IP / UDP / DNS Qry b'wpad.'
Ether / IP / UDP / DNS Ans name-error
Ether / IP / UDP / DNS Ans b'chat.freenode.net.'
Ether / IP / UDP / DNS Qry b'cr-tools.clients.google.com.'
Ether / IP / UDP / DNS Qry b'cr-tools.clients.google.com.'
Ether / IP / UDP / DNS Ans b'clients.l.google.com.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Qry b'wpad.'
Ether / IP / UDP / DNS Qry b'wpad.'
Ether / IP / UDP / DNS Ans name-error
Ether / IP / UDP / DNS Ans b'chat.freenode.net.'
Ether / IP / UDP / DNS Qry b'cr-tools.clients.google.com.'
Ether / IP / UDP / DNS Qry b'cr-tools.clients.google.com.'
Ether / IP / UDP / DNS Ans b'clients.l.google.com.'
Ether / IP / UDP / DNS Qry b'dl.javafx.com.'
Ether / IP / UDP / DNS Qry b'dl.javafx.com.'
Ether / IP / UDP / DNS Ans 137.254.16.78
Ether / IP / UDP / DNS Qry b'www.google.com.'
Ether / IP / UDP / DNS Qry b'www.google.com.'
Ether / IP / UDP / DNS Ans b'www.l.google.com.'
Ether / IP / UDP / DNS Qry b'www.nmap.org.'
Ether / IP / UDP / DNS Qry b'www.nmap.org.'
Ether / IP / UDP / DNS Ans 74.207.254.18
Ether / IP / UDP / DNS Qry b'nmap.org.'
Ether / IP / UDP / DNS Qry b'nmap.org.'
Ether / IP / UDP / DNS Ans 74.207.254.18
Ether / IP / UDP / DNS Qry b'wpad.'
Ether / IP / UDP / DNS Qry b'wpad.'
```

Checkpoint 1 — Write short answers:

- How many TCP, TCP-port-80, and UDP-port-53 packets did you find?
    a. TCP packets: 474035 found
    b. TCP port 80 packets: 22430 found
    c. UDP port 53 packets: 57 found
- Why might a filter return zero packets?
    a. A filter might return zero packets if the captured packets traffic does not match the filter requirements
- What is the difference between filtering by protocol and filtering by port?
    a. Filtering by protocol would filter the packets based on the protocol they use (ex. from lab TCP, UDP)
    b. Filtering by port would filter the packets based on the port they use (ex. from lab 80, 53)

Part 2


Command Prompt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 4>python scapy_test.py
PART 2
###[Ethernet]###
dst = 08:00:27:b5:b7:19
src = 00:1e:49:db:19:c3
type = ARP
###[ARP]###
hwtype = Ethernet (10Mb)
ptype = IPv4
hwlen = 6
plen = 4
op = who-has
hwsrc = 00:1e:49:db:19:c3
psrc = 147.32.84.1
hwdst = 08:00:27:b5:b7:19
pdst = 147.32.84.165
###[Padding]###
load = b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
Contains IPv4? False
Source IP: 78.40.125.4
Destination IP: 147.32.84.165
78.40.125.4 -> 147.32.84.165
147.32.84.165 -> 78.40.125.4
147.32.84.165 -> 78.40.125.4
78.40.125.4 -> 147.32.84.165
78.40.125.4 -> 147.32.84.165
TCP flags: PA
Size (bytes): 60
Timestamp: 1313153171.504306
Source: 78.40.125.4 Destination: 147.32.84.165 Bytes: 83 Time: 1313153244.357711
Source: 147.32.84.165 Destination: 78.40.125.4 Bytes: 83 Time: 1313153244.358073
Source: 147.32.84.165 Destination: 78.40.125.4 Bytes: 83 Time: 1313153244.358085
Source: 78.40.125.4 Destination: 147.32.84.165 Bytes: 60 Time: 1313153244.388656
Source: 78.40.125.4 Destination: 147.32.84.165 Bytes: 120 Time: 1313153273.62872
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 4>

Step 1

```
###[ Ethernet ]###
  dst       = 08:00:27:b5:b7:19
  src       = 00:1e:49:db:19:c3
  type      = ARP
###[ ARP ]###
     hwtype    = Ethernet (10Mb)
     ptype     = IPv4
     hwlen     = 6
     plen      = 4
     op        = who-has
     hwsrc     = 00:1e:49:db:19:c3
     psrc      = 147.32.84.1
     hwdst     = 08:00:27:b5:b7:19
     pdst      = 147.32.84.165
###[ Padding ]###
        load      = b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
```

Step 2

```
Contains IPv4? False
```

Step 3

```
Contains IPv4? False
Source IP: 78.40.125.4
Destination IP: 147.32.84.165
```

Step 4

```
78.40.125.4 -> 147.32.84.165
147.32.84.165 -> 78.40.125.4
147.32.84.165 -> 78.40.125.4
78.40.125.4 -> 147.32.84.165
78.40.125.4 -> 147.32.84.165
```

Step 5

```
78.40.125.4 ->
TCP flags: PA
```

Step 6

```
Size (bytes): 60
Timestamp: 1313153171.504306
```

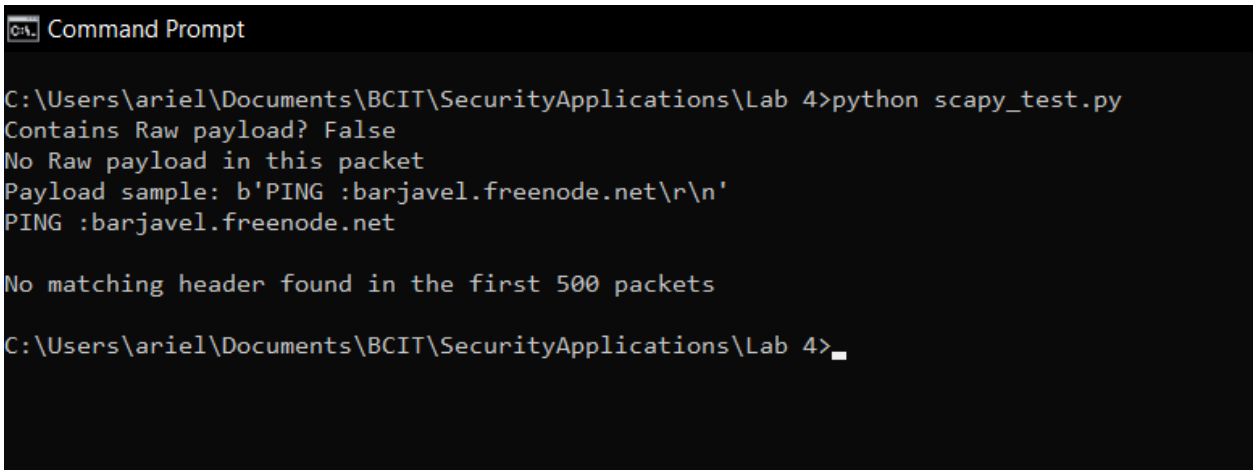
Step 7

```
Source: 78.40.125.4 Destination: 147.32.84.165 Bytes: 83 Time: 1313153244.357711
Source: 147.32.84.165 Destination: 78.40.125.4 Bytes: 83 Time: 1313153244.358073
Source: 147.32.84.165 Destination: 78.40.125.4 Bytes: 83 Time: 1313153244.358085
Source: 78.40.125.4 Destination: 147.32.84.165 Bytes: 60 Time: 1313153244.388656
Source: 78.40.125.4 Destination: 147.32.84.165 Bytes: 120 Time: 1313153273.62872
```

Checkpoint 2 — Answer:

- What do source and destination IP addresses tell you?
    a. The source and destination IP addresses show the endpoint machines trying to communicate with one another
- What is the purpose of TCP flags?
    a. TCP flags show the state of a TCP communication. In this test, the flags observed are P for PSH and A for ACK
- Why are timestamps useful when investigating possible flooding?
    a. Timestamps can help in measuring the traffic density. It shows when packets were captured and can help in distinguish network bursts or possible flooding

Part 3



Step 2

```
Contains Raw payload? False
```

Step 3

```
No Raw payload in this packet
```

Step 4

```
Payload sample: b'PING :barjavel.freenode.net\r\n'
```

Step 5

```
PING :barjavel.freenode.net
```

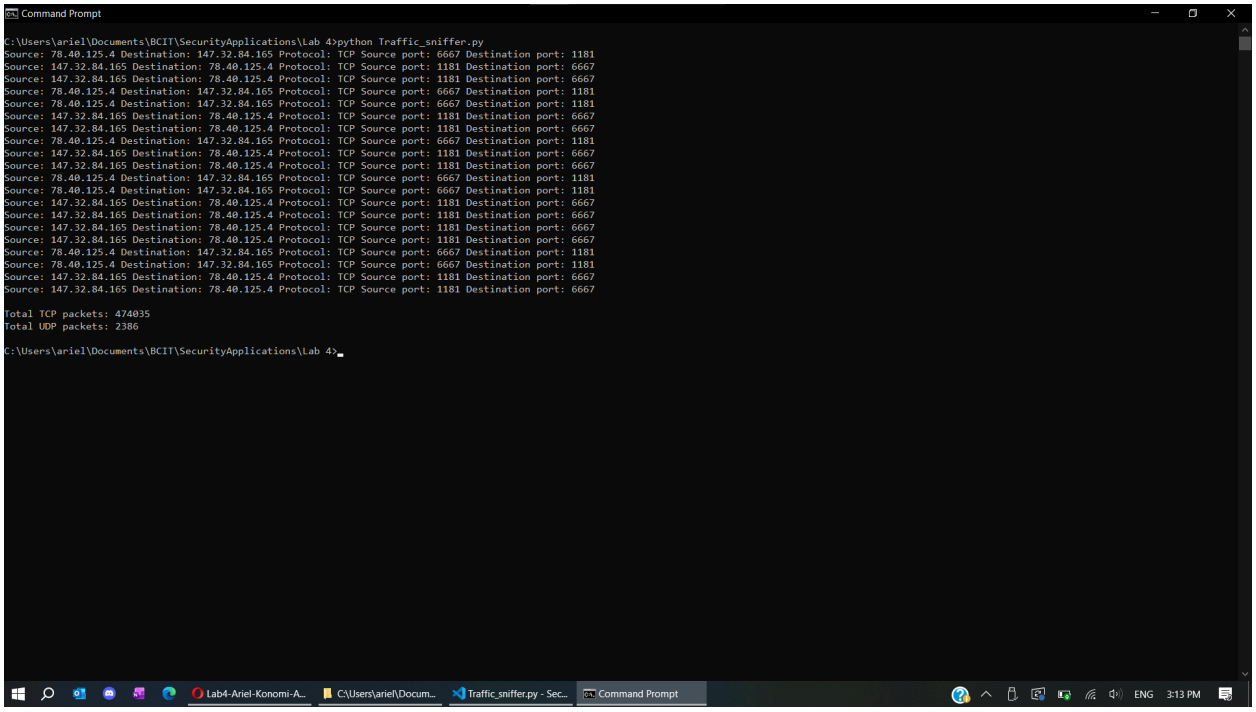
Step 6

```
No matching header found in the first 500 packets
```

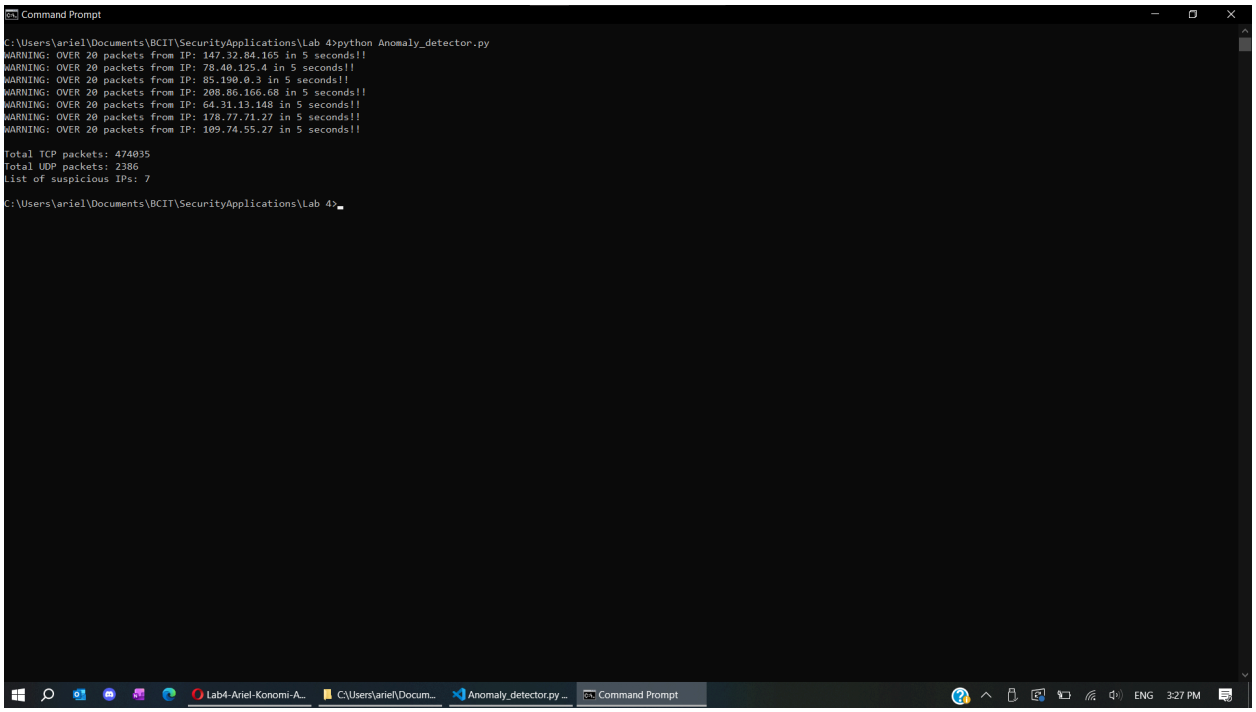
Checkpoint 3 — Answer:

- Did you find readable payload text?
    a. Yes, a readable payload text was found
        i. PING :barjavel.freenode.net
- What might plaintext HTTP expose?
    a. Plaintext HTTP might expose information such as HTTP headers and data being sent between the client and server
- Why can you not inspect the contents of encrypted HTTPS traffic using these simple commands?
    a. You cannot inspect the contents of encrypted HTTPS traffic using these simple commands as HTTPS encrypts the contents of the traffic

Part 4


Command Prompt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 4>python Traffic_sniffer.py
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Total TCP packets: 474035
Total UDP packets: 2386
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 4>
Lab4-Ariel-Konomi-A...
C:\Users\ariel\Docum...
Traffic_sniffer.py - Sec...
Command Prompt
ENG
3:13 PM

Part 5


Command Prompt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 4>python Anomaly_detector.py
WARNING: OVER 20 packets from IP: 147.32.84.165 in 5 seconds!!
WARNING: OVER 20 packets from IP: 78.40.125.4 in 5 seconds!!
WARNING: OVER 20 packets from IP: 85.190.0.3 in 5 seconds!!
WARNING: OVER 20 packets from IP: 208.86.166.68 in 5 seconds!!
WARNING: OVER 20 packets from IP: 64.31.13.148 in 5 seconds!!
WARNING: OVER 20 packets from IP: 178.77.71.27 in 5 seconds!!
WARNING: OVER 20 packets from IP: 109.74.55.27 in 5 seconds!!
Total TCP packets: 474035
Total UDP packets: 2386
List of suspicious IPs: 7
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 4>
Lab4-Ariel-Konomi-A...
C:\Users\ariel\Docum...
Anomaly_detector.py ...
Command Prompt
ENG
3:27 PM

Part 6 — Reflection questions

Answer briefly:

1. Why is packet inspection important for network security?
    a. Packet inspection is important for network security as it can be used to analyze network traffic for unusual activity and identify data exfiltration attempts
2. What types of security risks can arise from unencrypted traffic?
    a. A few types of security risks that can arise from unencrypted traffic would be: man in the middle attacks, session hijacking, eavesdropping and/or credential theft
3. What are the limitations of passive packet sniffing and PCAP analysis?
    a. Passive packet sniffing and PCAP analysis only analyze traffic that was already captured. It is unable to block attacks that happen in real time and is unable to read unencrypted traffic
4. How does encryption help prevent information leakage?
    a. Encryption helps prevent information leakage as it transforms readable cleartext data into protected data that would be unreadable by parties that are unauthorized. Encrypting this data prevents network monitoring
5. Why might a simple packet-rate detector produce false positives?
    a. A simple packet rate detector might produce false positives as many legitimate activities within the network might generate multiple packets within a very small time frame. Due to it being a simple packet rate detector, it cannot differentiate between malicious traffic or not

Part 7

In this lab, we were provided with a .pcap file which contained 495,056 packets. After a bit of testing with sample scripts, it was found that within all of these packets, 474,035 were TCP packets and 2,386 were UDP packets. During testing with these sample scripts, I was able to find communication using TCP traffic, which used port 80 and UDP traffic, which used port 53. These communications seemed to be using a few IP addresses, with the main one being 147.32.84.165. Later testing found PING commands being given and plaintext headers as well. A test of the first 500 packets was done to see if more information could be found, but no matching header was found. Another test showed high packet rates being generated for both TCP and UDP packets by multiple hosts during a 5 second period.

These first few tests and observations show security issues related to unencrypted network traffic. The main issue is that data is being transmitted without any encryption, making the information being transmitted easily readable by anyone trying to listen in and capture these

packets. The high packet rates in a short time frame could be malicious, but due to not having any information, nor a more complex script built, this all could be speculation of malice. There were about 7 IPs which exceeded the packet generation limit for both TCP and UDP packets within that 5 second period.

PCAP has a few limitations which are shown during this lab. The method of packet capture used in this lab operates after an event has happened and registered. This means that it cannot block any active attacks in real time. It also means that it would be unable to read any packets or traffic if they are encrypted. The rule set for the previous test of flagging IPs with 20 or more packets generated in 5 seconds is also a bit finicky and can create false positives.

To mitigate risks and vulnerabilities, administrators would need to enforce a few security changes. All network communications would need to be encrypted with TLS. Active intrusion prevention systems would also need to be implemented alongside PCAP, to ensure that the data and servers would remain secure in an active attack if it ever happened.